

Logging & Observability

Last updated: 2026-06-10

Structured server-side logging (Winston) shipped to a self-hosted Grafana Loki + Alloy + Grafana stack running alongside the app on the production EC2 instance. Logs never touch MongoDB Atlas.

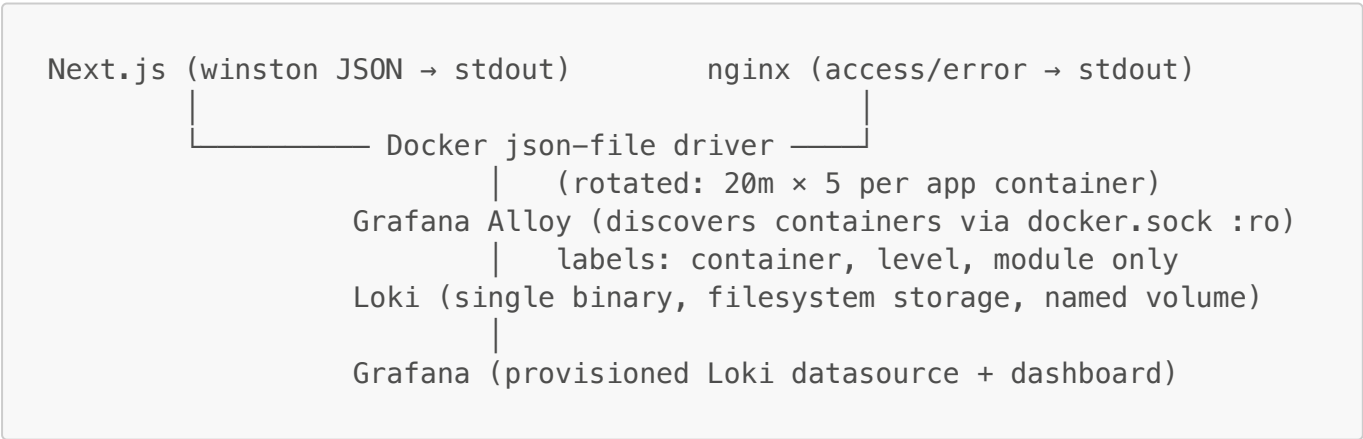
Dashboards

What	Where	Access
Grafana (log search + dashboards)	https://fakefourrecords.com/grafana/	Grafana login — user <code>admin</code> , password = <code>GRAFANA_ADMIN_PASSWORD</code> GitHub secret
Provisioned dashboard	Grafana → Dashboards → "Boudreaux — Application Logs"	Same

Anonymous access and sign-up are disabled (`GF_AUTH_ANONYMOUS_ENABLED=false`, `GF_USERS_ALLOW_SIGN_UP=false`). nginx proxies `/grafana/` over TLS with a Docker-DNS variable upstream, so the site stays up even if the logging stack is down.

Required GitHub secret: `GRAFANA_ADMIN_PASSWORD` must be created in the repository secrets before the next deploy. Optional repo variable `LOG_LEVEL` overrides the production log level (default `info`).

Architecture



- **Winston logger:** `src/lib/utils/logger.ts` — `createLogger(module) / loggers.*`. Production emits single-line JSON with `service`, `module`, `level`, `timestamp`, plus structured data (sensitive keys redacted).
- **Collector:** `observability/alloy/config.alloy`. Only `container`, `level`, `module` become Loki labels (bounded cardinality); everything else is queried with `| json`. nginx `/health` probes are dropped.
- **Store:** `observability/loki/loki.yml`, data on the `loki-data` named volume.
- **Error capture:** `src/instrumentation.ts` (`onRequestError`) logs uncaught server/RSC errors with the digest; `src/app/error.tsx` + `src/app/global-error.tsx` report client-render errors

to `POST /api/client-errors` (rate-limited 5/min/IP, 2 KB cap, Zod-validated, no stacks or PII).

Useful LogQL queries

```
{container="website", level="error"} # all server errors
{container="website"} | json | module="AUDIT" # security audit events
{container="website"} |= "Rate limit exceeded" # 429s
{container="website", module="PAYMENTS"} # checkout flow
sum by (module) (count_over_time({container="website", level="error"}[5m]))
```

Retention & volume limits

Layer	Limit
Loki retention	30 days (compactor, <code>retention_period: 720h</code>)
Loki ingest cap	4 MB/s (burst 8 MB), 1 MB/s per stream
Docker json-file (website)	20 MB × 5 files = 100 MB max
Docker json-file (nginx)	20 MB × 5 files = 100 MB max
Docker json-file (log stack)	10 MB × 3 files = 30 MB max per container
Success-log sampling	1-in-20 (<code>withLogging</code>), 1-in-10 download confirms

Expected steady-state disk usage is well under 2 GB. A hot error loop hits Loki's per-stream limit and drops excess lines (designed backpressure); the rotated Docker json-files remain available for `docker logs` forensics. `deploy-remote.sh` intentionally prunes **without** `--volumes` so `loki-data` and `grafana-data` survive low-disk cleanups.

Resource budget (t3.large: 2 vCPU / 8 GB)

Container	CPU limit	Memory limit
website	2.0	2048 M
loki	0.5	512 M
grafana	0.5	384 M
alloy	0.25	256 M

Developer experience

```
SHOW_DEV_LOGS=true pnpm dev # colored, keyed, human-readable logs
pnpm dev # quiet (warn+ only)
```

Env var	Effect
<code>SHOW_DEV_LOGS</code>	<code>true</code> → debug-level colored dev output
<code>LOG_LEVEL</code>	<code>debug info warn error</code> override (prod default <code>info</code>)
<code>LOG_DEBUG_MODULES</code>	Comma list (e.g. <code>CHAT,PRESIGNED_URLS</code>) forced to debug

Runtime level override (admin)

Admins can flip the global level at runtime without a restart — e.g. enable `debug` during an incident:

- **UI:** <https://fakefourrecords.com/admin/logging> (admin session required; reachable from the admin dashboard dropdown). Pick a level + duration (15 min / 1 h / 4 h), or "Reset to default".
- **State check:** `GET /api/admin/log-level` → `{ configuredLevel, override, effectiveLevel, expiresAt }`.
- **Semantics:** overrides auto-revert after their TTL (default **60 min**), and always reset on deploy/container restart (in-memory, single container). The `AUDIT` module never goes coarser than `info`, so security events keep flowing even at a `warn/error` override. Every change is itself audited (`admin.log_level.changed`).
- **Code:** `setRuntimeLogLevel/getLogLevelState` in `src/lib/utils/logger.ts`; mutation via `setLogLevelAction` (`src/lib/actions/set-log-level-action.ts`).

Conventions:

- Reuse `loggers.<area>` or `createLogger('MODULE')`; never `console.*` in server code.
- Wrap new API routes with `withLogging('MODULE')(handler)` (outermost) and Server Actions with `logAction(...)` from `src/lib/decorators/with-logging.ts`.
- Sample high-frequency success logs with `shouldSample(key, n)`; always log warns/errors.
- Never log secrets/PII. `safeSerialize` redacts keys containing `password`, `secret`, `token`, `key`, `session`, etc. — note a Stripe checkout session id must be logged as `checkoutId` (a `sessionId` key would be redacted).

Why Loki instead of ELK

Elasticsearch needs ~2 GB JVM heap (~4 GB container) plus Kibana ~1 GB — unworkable next to the app's 2 GB cap on an 8 GB t3.large. Loki indexes only labels (not full text), stores compressed chunks on the local filesystem, and the whole stack fits in ~1.15 GB with hard limits. Grafana replaces Kibana; Alloy replaces Filebeat/Logstash. Tradeoff: LogQL label queries + grep rather than a full-text inverted index — no practical loss at this app's log volume.

Licensing / cost

Component	Version	License	Cost
winston	3.x	MIT	\$0
Grafana Loki	3.7.2	AGPL-3.0	\$0 (self-hosted)
Grafana OSS	12.4.3	AGPL-3.0	\$0 (self-hosted)

Component	Version	License	Cost
Grafana Alloy	v1.16.3	Apache-2.0	\$0

All components run as unmodified network services — AGPL imposes no obligations on the app code. No SaaS, no free-tier ceilings, no per-GB fees.

Security notes

- Alloy mounts `/var/run/docker.sock` **read-only**, with `cap_drop: ALL, no-new-privileges`, a read-only rootfs, and no published ports. Follow-up hardening option: front the socket with a `docker-socket-proxy` container.
- Loki and Grafana are only **exposed** on the compose network — never published to the host. The single public entry point is nginx's `/grafana/` location.
- All stack containers run read-only rootfs with named volumes for state, matching the app's hardening posture.

Local smoke test

```
docker compose -f docker-compose.prod.yml -f docker-compose.logging.yml
config -q # validate
docker run --rm -v "$PWD/observability/loki/loki.yml:/etc/loki/loki.yml:ro" \
\
  grafana/loki:3.7.2 -config.file=/etc/loki/loki.yml -verify-config
docker run --rm -v
"$PWD/observability/alloy/config.alloy:/etc/alloy/config.alloy:ro" \
  grafana/alloy:v1.16.3 validate /etc/alloy/config.alloy
```